

# NP-Hardness of (3,4)-SAT and [2,3]-Bounded 3-SAT

Yuval Itzhaki

Claude

## Abstract

Tovey [4] showed that satisfiability remains NP-hard when every clause has exactly three literals and every variable occurs at most four times, a restriction that has become a standard source problem for hardness reductions. This note presents a formalization of that theorem in Lean, together with a second theorem that scheduling reductions use more often: satisfiability remains NP-hard when every clause has two or three literals and every literal occurs at most twice. Both theorems are proved by polynomial-time many-one reductions from unrestricted satisfiability, each of which is stated in three parts: the construction, its correctness, and its running time on the word RAM. The note gives the constructions as they are formalized and links each of them to its Lean counterpart.

## 1 Introduction

Occurrence-bounded variants of satisfiability are among the most frequently used starting points for NP-hardness proofs. Tovey [4] proved that deciding satisfiability is NP-hard even for formulas in which every clause has exactly three literals and every variable occurs in at most four clauses (Theorem 2.3 of that paper). We call this problem (3,4)-SAT. It is the source of the reduction for interval scheduling with eligible machine sets of Hermelin, Itzhaki, Molter and Shabtay [3]. The scheduling problems of Heeger, Hermelin, Itzhaki, Molter and Shabtay [2] start instead from a form of the theorem in which clauses have two or three literals and every *literal*, a variable together with its sign, occurs at most twice. We call this problem [2,3]-BOUNDED 3-SAT.

This note presents the Lean formalization of both theorems. The theorems are stated in Section 3. Each is proved by a reduction, and each reduction is formalized in the same three parts: the construction as a definition (Constructions 1 and 2), the statement that it is correct (Lemmas 1 to 3 and 5 to 8), and the statement that it runs in polynomial time (Lemmas 4 and 9). The reductions are plain polynomial-time many-one reductions, so there is no parameter whose growth would need a further bound.

## 2 Preliminaries

A *literal* is a variable  $x_i$  or its negation  $\bar{x}_i$ . A *clause* is a list of literals and a *CNF formula* is a list of clauses; a literal may be repeated within a clause. The empty clause is unsatisfiable. A *truth assignment* assigns a truth value to every variable, it *satisfies* a formula if every clause contains a literal made true by it, and a formula is *satisfiable* if some assignment satisfies it. CNF formulas are written as binary words by the encoding of the Cook–Levin theorem as formalized in `lax-429075`, and SATISFIABILITY is the language of the words that encode satisfiable formulas. By the Cook–Levin theorem [1], every language in NP has a polynomial-time many-one reduction to SATISFIABILITY. A *polynomial-time many-one reduction* from a language  $A$  to a language  $L$  is a function on words that is computable in polynomial time by a Turing machine and maps the words of  $A$  exactly to the words of  $L$ , and a language is *NP-hard* if every language in NP has such a reduction to it. The two theorems below follow by composing the Cook–Levin reduction with the reductions of this note.

The classes P and NP are defined by Turing machines, as formalized in `1ax-434930`, and polynomial-time computable functions are those computed by a Turing machine within a polynomial number of steps.

Running times are proved for the word RAM: a program is a fixed list of instructions operating on words of  $w$  bits, and its running time is the number of instructions it executes. A statement about a running time is a statement about one program, at every word length that admits the input. Polynomial time on the word RAM and polynomial time on Turing machines coincide, as formalized in `1ax-759944`. Each running-time statement below is therefore stated for Turing machines and proved by a word RAM program. The programs read their input with the reader and the scanner of CNF encodings of the formalization of Itzhaki and Claude (`1ax-391470`), which also supplies the transfer from the word RAM to Turing machines.

### 3 The Two Problems

**Definition 1** (The Problem (3,4)-SAT). A CNF formula is a  $(3,4)$  *formula* if every clause has exactly three literals and every variable occurs at most four times, counting the occurrences in all clauses, with multiplicity. (3,4)-SAT is the language of the encodings of satisfiable  $(3,4)$  formulas.

**Theorem 1** (Theorem 2.3 of [4]). *(3,4)-SAT is NP-hard.*

**Definition 2** (The Problem [2,3]-BOUNDED 3-SAT). A  $[2,3]$ -*bounded 3-SAT formula* is a conjunction of clauses of two literals and of clauses of three literals in which every literal occurs at most twice, so every variable occurs at most twice positively and at most twice negatively. Its *positions* are the places for literals inside clauses, two per clause of two literals and three per clause of three literals.

The formulas are written as binary words in a self-delimiting code. A number is written as its binary digits, least significant first, preceded by their count in unary and a zero. A formula is the number of its variables, the numbers of its clauses of two and of three literals, and then the variable and the sign of each literal of each clause, the clauses of two literals first. [2,3]-BOUNDED 3-SAT is the language of the words that encode satisfiable formulas that have at most as many variables as positions. Every formula in which each variable occurs is of this kind, so no formula is excluded by the condition. It is needed because the number of variables is written in binary: a word could name exponentially many variables without using them, and a reduction that writes something for each variable could not run in polynomial time.

**Theorem 2.** *[2,3]-BOUNDED 3-SAT is NP-hard.*

The two theorems are stated for different encodings. (3,4)-SAT uses the encoding of formulas of the Cook–Levin theorem, so that it is a restriction of SATISFIABILITY on the same words. [2,3]-BOUNDED 3-SAT uses the self-delimiting code, which is the code read by the scheduling formalizations that start from it.

### 4 From Satisfiability to (3,4)-SAT

**Construction 1** (From CNF Formulas to  $(3,4)$  Formulas). Let  $\varphi$  be a CNF formula with  $N$  literal occurrences over variables  $x_0, \dots, x_{V-1}$ . The occurrences are numbered  $0, \dots, N-1$  in the order in which they appear, clause by clause; this number is the *position* of the occurrence. The construction has two steps, which use a gadget.

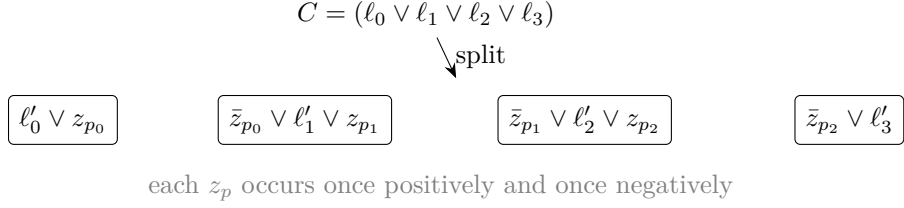


Figure 1: Splitting a clause of four literals. The literal  $\ell'_i$  is  $\ell_i$  on the copy of its variable at its position. The chain can be satisfied exactly when  $C$  can, because the links propagate “no literal so far is true” from the first clause to the last.

*The gadget.* For a variable  $y$  and nine further variables  $a_j, b_j, d_j$  for  $j \in \{0, 1, 2\}$ , the gadget  $G(y)$  consists of the thirteen clauses

$$\begin{aligned}
& (y \vee a_j \vee b_j) && \text{for } j \in \{0, 1, 2\}, \\
& (d_j \vee a_j \vee \bar{b}_j), (d_j \vee \bar{a}_j \vee b_j), (d_j \vee \bar{a}_j \vee \bar{b}_j) && \text{for } j \in \{0, 1, 2\}, \\
& (\bar{d}_0 \vee \bar{d}_1 \vee \bar{d}_2).
\end{aligned}$$

The gadget is satisfiable, and it forces  $y$  to be true. The variable  $y$  occurs three times in it and each of the other nine variables four times.

*Splitting.* Every occurrence at position  $p$  of a variable  $x_v$  is replaced by a fresh variable  $w_{v,p}$  with the same sign. A clause  $C = (\ell_0 \vee \dots \vee \ell_{L-1})$  whose literals sit at the positions  $p_0, \dots, p_{L-1}$  is replaced by the chain of clauses

$$(\ell'_0 \vee z_{p_0}), (\bar{z}_{p_0} \vee \ell'_1 \vee z_{p_1}), \dots, (\bar{z}_{p_{L-2}} \vee \ell'_{L-1}),$$

where  $\ell'_i$  is  $\ell_i$  on the copy of its variable at position  $p_i$  and  $z_p$  is a fresh *link* variable for each position  $p$ . A clause of one literal becomes that literal alone and an empty clause stays empty. Finally, for every  $v < V$  and every position  $p < N$  we add the clause  $(w_{v,p} \vee \bar{w}_{v,p+1})$ , that is the implication  $w_{v,p+1} \rightarrow w_{v,p}$ , where  $p+1$  is taken modulo  $N$  (see Figures 1 and 2). The result has clauses of at most three literals and every variable occurs at most three times.

*Padding.* Let  $B$  exceed all variables used so far. The  $\kappa$ -th clause  $C'$  of the split formula receives three private gadgets  $G(y_{\kappa,0}), G(y_{\kappa,1}), G(y_{\kappa,2})$  over fresh variables above  $B$ , and is padded to three literals by appending  $\bar{y}_{\kappa,t}$  for the first  $3 - |C'|$  values of  $t$ . The result is the  $(3, 4)$  formula  $T(\varphi)$ .

On words, the reduction reads a word, decodes the formula it encodes, applies  $T$  to it, and encodes the result. A word that encodes no formula is treated as the formula that consists of one empty clause.

All copies  $w_{v,p}$  with  $v < V$  and  $p < N$  are created and tied together, and not only those of pairs  $(v, p)$  that occur. This makes the cyclic successor of a position simply the next position, at the price of an output of quadratic size.

**Lemma 1.** *For every CNF formula  $\varphi$ ,  $T(\varphi)$  is a  $(3, 4)$  formula.*

*Proof idea.* Padding gives every clause exactly three literals. A variable below  $B$  has the occurrences that it has in the split formula, at most three. Above  $B$  a variable belongs to one gadget of one clause: it occurs at most four times in the gadget, and the forced variable occurs three times in it and once more as padding.  $\square$

**Lemma 2.** *A CNF formula  $\varphi$  is satisfiable if and only if  $T(\varphi)$  is satisfiable.*

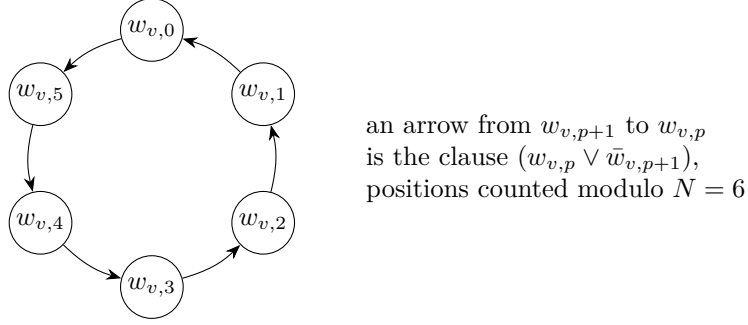


Figure 2: The copies of one variable  $x_v$  over  $N = 6$  positions. A satisfying assignment has to give all copies the same value, since the implications form a cycle.

*Proof idea.* Given an assignment of  $\varphi$ , set every copy  $w_{v,p}$  to the value of  $x_v$ , and set the link  $z_p$  to true exactly if no literal of its clause up to position  $p$  is true. This satisfies the chains and the implications. Conversely, the implications form a cycle for every variable, so all copies of  $x_v$  agree and give an assignment of  $\varphi$ . If it made all literals of a clause false, the links would propagate true values from the first clause of its chain to the last, which would then be falsified. For the padding, the gadget  $G(y)$  is satisfiable with  $y$  true and unsatisfiable with  $y$  false: if  $y$  is false then each pair  $a_j, b_j$  has a true member, which forces  $d_j$  to be true for all  $j$  and falsifies the last clause. So every padding literal  $\bar{y}_{\kappa,t}$  is false, and a padded clause is satisfied exactly when the clause before padding is.  $\square$

**Lemma 3.** *For every word  $w$ ,  $w \in \text{SATISFIABILITY}$  if and only if the reduction maps  $w$  to a word of (3,4)-SAT.*

*Proof idea.* The encoding of formulas is injective and a word that encodes a formula is decoded to it. So for such a word the statement is Lemmas 1 and 2. A word that encodes no formula is not in SATISFIABILITY, and its image is the image of the unsatisfiable formula that consists of one empty clause.  $\square$

**Lemma 4.** *The reduction of Construction 1 is computable in polynomial time by a Turing machine.*

*Proof idea.* A word RAM program computes the reduction on the zeros and ones of its input. A scan decodes the formula into arrays of variables, signs and clause numbers, one pass tabulates where each clause ends, and the chains are then written clause by clause, and the cycle clauses position by position, each with its padding and its three gadgets. The output has at most quadratically many clauses, and the program runs within a polynomial of degree four in the number of bits of the input. Polynomial time on the word RAM transfers to Turing machines.  $\square$

*Proof of Theorem 1.* By the Cook–Levin theorem, every language in NP reduces to SATISFIABILITY in polynomial time. By Lemmas 3 and 4 the reduction of Construction 1 is a polynomial-time many-one reduction from SATISFIABILITY to (3,4)-SAT, and polynomial-time many-one reductions compose.  $\square$

## 5 From (3,4)-SAT to [2,3]-Bounded 3-SAT

**Construction 2** (From (3,4) Formulas to [2,3]-Bounded Formulas). Let  $\varphi$  be a CNF formula with  $C$  clauses, each of three literals, over variables below  $V$ . It has  $N = 3C$  positions: position  $p = 3c + t$  holds the  $t$ -th literal of clause  $c$ . Every variable  $x_v$  with  $v < V$  gets a copy  $x_{v,p}$  at every position  $p < N$ , so the result has  $V \cdot N$  variables; the clauses of three literals use only the copy at position  $p$  of the variable that occurs there. The result consists of two kinds of clauses.

- The  $C$  clauses of three literals are the clauses of  $\varphi$ , with the literal at position  $p$  replaced by the copy for position  $p$  and keeping its sign.
- The  $V \cdot N$  clauses of two literals are the implications  $x_{v,p+1} \rightarrow x_{v,p}$ , that is  $(x_{v,p} \vee \bar{x}_{v,p+1})$ , for all  $v < V$  and all positions  $p < N$ , with positions taken cyclically (see Figure 2). They are the cycle clauses of Construction 1.

Every literal occurs at most twice. A copy belongs to one position only, so it appears in at most one clause of three literals, and with one sign. In the cycle of its variable it is the consequent of one implication and the antecedent of another, which gives it one positive and one negative occurrence there. So one of its two literals occurs once, and the other at most twice.

On words, the reduction is the reduction of Construction 1 followed by this construction: a word is sent to the encoding of a  $(3, 4)$  formula, that formula is decoded, and the construction is applied to it and written in the self-delimiting code.

A  $[2, 3]$ -bounded formula carries its occurrence bound as part of the data, so the formalization defines the construction as the formula above when the copies meet the bound and as the empty formula otherwise. Lemma 5 states that the bound is always met, so the second case never arises. The construction alone reads any word, but it is correct only for formulas whose clauses all have three literals, which is all that Construction 1 produces; correctness on words is therefore stated for the composition.

**Lemma 5.** *In the formula of Construction 2, every literal occurs at most twice.*

*Proof idea.* Two positions that carry the same literal lie on the same side, among the cycle clauses or among the clauses of three literals, because the sign pattern differs. On either side the copy map and the successor map of the cycle are injective, so the two positions coincide.  $\square$

**Lemma 6.** *The formula of Construction 2 has at most as many variables as positions.*

*Proof idea.* There are  $V \cdot N$  variables, and the  $V \cdot N$  clauses of two literals have  $2VN$  positions already.  $\square$

**Lemma 7.** *Let  $\varphi$  be a CNF formula all of whose clauses have three literals. Then  $\varphi$  is satisfiable if and only if the formula of Construction 2 is satisfiable.*

*Proof idea.* Given an assignment of  $\varphi$ , every copy takes the value of its variable. Conversely, the cycle clauses force all copies of a variable to agree, so reading the value of  $x_v$  off any copy gives an assignment of  $\varphi$ , and the clause on the copies of the literals of a clause  $c$  is satisfied exactly when  $c$  is.  $\square$

**Lemma 8.** *For every word  $w$ ,  $w \in \text{SATISFIABILITY}$  if and only if the reduction of Construction 2 maps  $w$  to a word of  $[2, 3]\text{-BOUNDED 3-SAT}$ .*

*Proof idea.* By Lemma 3 and Lemma 1, the first step maps  $w$  to the encoding of a formula that is satisfiable exactly when  $w \in \text{SATISFIABILITY}$ , and all of whose clauses have three literals. Lemmas 5 to 7 then apply to this formula. The self-delimiting code is prefix-free, so a word in  $[2, 3]\text{-BOUNDED 3-SAT}$  is the encoding of the formula that the construction builds from it, and the statement follows.  $\square$

**Lemma 9.** *The second step of the reduction, from the encoding of a formula to the encoding of its  $[2, 3]$ -bounded formula, is computable in polynomial time by a Turing machine.*

*Proof idea.* A word RAM program reuses the scan of the first program, which decodes the formula into arrays. It writes the three numbers of the header, then the  $V \cdot N$  cycle clauses by one loop, then the copies of the three literals of every clause by a second loop, each number in the self-delimiting code by three short loops. The program runs within a polynomial of degree four in the number of bits of the input, and polynomial time on the word RAM transfers to Turing machines.  $\square$

*Proof of Theorem 2.* By the Cook–Levin theorem, every language in NP reduces to SATISFIABILITY in polynomial time. By Lemmas 3 and 4 the first reduction is a polynomial-time many-one reduction to (3,4)-SAT, and by Lemmas 4, 8 and 9 its composition with the second step is a polynomial-time many-one reduction from SATISFIABILITY to [2,3]-BOUNDED 3-SAT. Polynomial-time many-one reductions compose.  $\square$

## References

- [1] S. A. Cook. The Complexity of Theorem-Proving Procedures. In *Proceedings of the Third Annual ACM Symposium on Theory of Computing (STOC 1971)*, pages 151–158. ACM, 1971. DOI: [10.1145/800157.805047](https://doi.org/10.1145/800157.805047)
- [2] K. Heeger, D. Hermelin, Y. Itzhaki, H. Molter and D. Shabtay. Fair Repetitive Interval Scheduling. *Algorithmica*, 2025. DOI: [10.1007/s00453-025-01322-y](https://doi.org/10.1007/s00453-025-01322-y)
- [3] D. Hermelin, Y. Itzhaki, H. Molter and D. Shabtay. On the Parameterized Complexity of Interval Scheduling with Eligible Machine Sets. *Journal of Computer and System Sciences* 144:103533, 2024. DOI: [10.1016/j.jcss.2024.103533](https://doi.org/10.1016/j.jcss.2024.103533)
- [4] C. A. Tovey. A Simplified NP-Complete Satisfiability Problem. *Discrete Applied Mathematics* 8(1):85–89, 1984. DOI: [10.1016/0166-218X\(84\)90081-7](https://doi.org/10.1016/0166-218X(84)90081-7)